

12.7 Text-to-SQL

The internet is teeming with examples that use an LLM to filter a database with one table and five columns to the largest value (“Which department generated the most sales?”). It’s hardly surprising that an LLM to which the table structure is passed as a context can formulate a corresponding SQL query. With real-life data, things look a little different.

In the following sections, we’ll look at the dataset of the viel-falter project. In this research project, which we’ve already used for content in the first part of this chapter, butterflies are identified according to their species in designated areas.

12.7.1 Database Export from MongoDB to SQLite

The data for the project has been stored in MongoDB, a NoSQL database, for more than 10 years. At MongoDB, efforts have been underway for some time to make the database content searchable using AI technology, but this mainly concerns the cloud offerings (MongoDB Atlas) and not the open-source version. For our example, we wanted to test the capabilities of current LLMs in dealing with SQL databases.

As NoSQL databases have structures such as arrays or nested documents, exporting to an SQL database isn’t easy. However, with the knowledge of which data is to be exported to which tables, we (authors and AI) can create an export script that takes over this function.

We’ve printed excerpts of a sample document from the MongoDB surveys collection here:

```
{
  "_id": {
    "$oid": "669e162e2a7d5ed3d645257d"
  },
```

```

"start": "13:52",
"end": "14:28",
"site": {
  "_id": {
    "$oid": "5e60d1d095c061fecc14670f"
  },
  "name": "132_alm_1",
  "community": "Lech",
  "nuts2": "Vorarlberg",
  "stratum": "alm",
  "climaticRegion": "A",
  "geometry": {
    "type": "Point",
    "coordinates": [10.159735679626467, 47.21806151908246]
  }
},
"notes": "Maculinea arion ausserhalb Erhebung",
"speciesList": [
  {
    "id": "595df3ce912bcbcc8f70765d",
    "count": 2
  },
  {
    "id": "595df3ce912bcbcc8f70769c",
    "count": 1
  }
],
"lastMod": {
  "$date": "2024-07-22T08:19:58.232Z"
},
"date": {
  "$date": "2024-07-21T11:52:00.000Z"
}
...
}

```

The final prompt we used to make Claude 3.5 Sonnet export parts of the MongoDB database to an SQLite database was as follows:

Prompt: Write a Python script to export a MongoDB collection surveys from a database `vielfalter` to SQLite. Extract the array field `speciesList` to a sepa-

rate table `speciesList`. Note that this is an array of documents, with an `id` and `count` field.

Extract the embedded documents in `site` to a separate table, including fields `_id`, `name`, `community`, and `nuts2`, removing duplicates and linking it to `surveys`. Include fields `_id`, `date`, `start`, `end`, and `notes` for the `surveys` table.

Export the collection `species` to a table including fields `_id`, `german`, and `latin`.

The resulting Python script was executable without modification, connected to the locally running copy of the database and exported the data to an SQLite database.

However, we spent several hours tweaking this prompt. Originally, we wanted to create a generic export script that would automatically recognize the arrays and embedded documents and create the SQLite tables accordingly. However, we were unable to wrap these requirements into a single prompt.

What worked surprisingly well, however, was an iterative approach in which we further developed the script in a chat with the LLM. With a little manual work at the end (we simply ran out of patience for the dialogs with the AI tool), we came to a usable result with the following prompts.

Prompt: How do I export a mongo collection to CSV where arrays in the collection get separate CSV files with foreign keys?

Interestingly, our results were better when we exported to comma-separated values (CSV) first and brought SQLite into play later. Unfortunately, the

first script we received in response to this prompt wasn't executable. Claude pointed out the error and was able to rectify it.

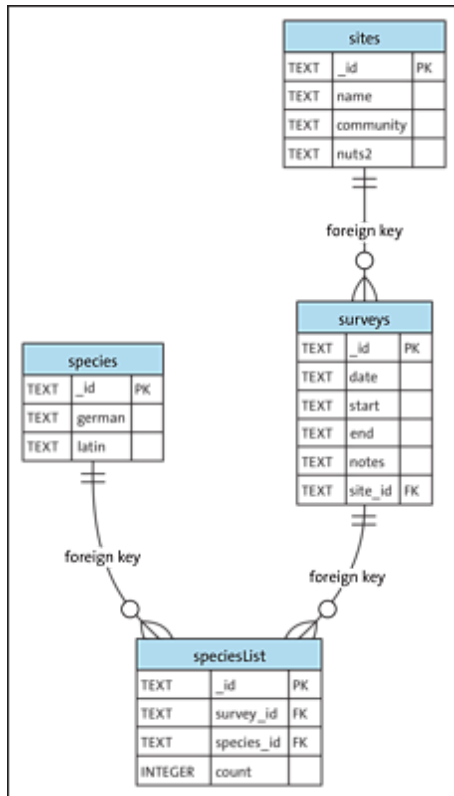


Figure 12.4 The Entity-Relationship Diagram of the Exported SQLite Database

Prompt: Python returns an error for this script:

```
Traceback (most recent call last):
  File "/data/export.py", line 31, in <module>
    writer.writerow(['parent_id'] +
                    first_doc[array_field][0].keys())
TypeError: can only concatenate list (not "dict_keys")
to list
```

The CSV export worked, but we had JSON strings in the exported CSV files, which were originally arrays of documents in our collection.

Prompt: Great! Can you improve a little bit, so that array fields aren't JSON strings but separate columns in CSV files?

Claude did this with flying colors. The field names of the documents were recognized and created as columns in CSV. Next, we wanted to store the em-

bedded document for `sites` in a separate CSV file, as is logical in an SQL database according to the normal forms.

Prompt: Perfect. What would I need to do if I want to normalize one key of the document, where the value holds an embedded document?

As the embedded documents often describe the same `sites`, we wanted to avoid duplicates.

Prompt: Great. Can you improve a bit more, so that we don't get duplicate entries in the new CSV file with the embedded document?

Claude was also able to solve this task. Although the embedded MongoDB ID wasn't used for comparison, but a hash was created using the document, the approach also works. Now we wanted to create an SQLite database instead of CSV files.

Prompt: Thanks, this works perfectly. Can you alter the script to write the data to an SQLite database instead of CSV files?

And lo and behold, Claude was able to correctly add the necessary libraries to the script and write the export to an SQLite file. During the many tests, we always had to adjust the collection name for each test. To avoid this, we instructed the LLM to use the correct name in the script. We also needed the `species` collection in the SQLite database, which contains the list of all butterflies, just like in a relational database.

Prompt: Excellent. Can you rename the exported collection `main_collection` to table `surveys` and export another collection named `species` from the same Mongo database to the same SQLite database?

The resulting Python code, which we haven't printed here, is quite readable, even if a Python expert would probably have designed one or two functions more efficiently. The result was sufficient for our purposes. Would we have been faster if we had written the export script ourselves? We can answer this question in the affirmative.

12.7.2 Text-to-SQL with LlamaIndex

We can make the exported SQLite database available to the LlamaIndex library without hesitation because the production data remains in the MongoDB, to which the script has no access. Of course, you can also connect a productive SQL database here instead of an export. The SQLAlchemy library we used supports common relational databases such as MySQL/MariaDB, PostgreSQL, Microsoft SQL Server, and Oracle. The problem is that you don't know in advance which SQL statements the AI tool will create. Even if the AI tool only has read-only access to the tables, a database administrator will probably roll their eyes at the prospect of uncontrolled SQL queries.

The LlamaIndex library makes Text-to-SQL queries child's play. The `NLSQLTableQueryEngine` module generates the *QueryEngine* that we use for queries in natural language. An *SQLAlchemy engine* with the reference to the database file is created beforehand. You must also install this Python



```
from llama_index.core import (
    Settings,
    SQLiteDatabase,
)
from llama_index.core.query_engine import
NLSQLTableQueryEngine
from llama_index.llms.anthropic import Anthropic
from sqlalchemy import create_engine

tables = ["surveys", "speciesList", "species", "sites"]
engine = create_engine("sqlite:data/vielfalter.db")
sql_database = SQLiteDatabase(
    engine,
    include_tables=tables
)
query_engine = NLSQLTableQueryEngine(
    sql_database=sql_database,
    tables=tables
)
Settings.llm = Anthropic(model="claude-3-5-sonnet-
20240620")
```

```

response = query_engine.query(
    "How many species were found in 2024?"
)
print(response)

```

To answer the final question about the number of different species found in 2024, the LlamaIndex library first reads the structure of the listed tables. As you can imagine, the naming of tables and columns is very important so that the LLM can later generate correct SQL commands.

```

INFO:llama_index.core.indices.struct_store.sql_retriever:>
  Table desc str: Table 'surveys' has columns: _id (TEXT),
    date (TEXT), start (TEXT), end (TEXT), notes (TEXT),
    site_id (TEXT),
    and foreign keys: ['site_id'] -> sites['_id'].

  Table 'speciesList' has columns: _id (TEXT), survey_id (TEXT),
    species_id (TEXT), count (INTEGER),
    and foreign keys: ['survey_id'] -> surveys['_id'],
    ['species_id'] -> species['_id'].
  ...

```

This information is sent to the LLM together with detailed instructions for creating the SQL query (we'll come to this in a moment), which in turn sends back an initial SQL suggestion:

```

Predicted SQL query:
SELECT COUNT(DISTINCT sl.species_id) AS species_count
FROM surveys s
JOIN speciesList sl ON s.`_id` = sl.survey_id
WHERE strftime('%Y', s.date) = '2024'

```

The LlamaIndex library then connects to the SQLite database and executes the query. The result gets included in a new prompt and is resent to the LLM as SQL Response: [(79,)]. The LLM formulates the short and correct answer (at the time of writing):

Based on the SQL query results, 79 distinct species were found in surveys conducted in 2024.

Even if this sounds like a very simple question, Claude has to demonstrate a bit of SQL know-how. The `speciesList` table must be linked to the `surveys` table and the date field must be analyzed correctly. The fact that we've formulated the prompt in English helps the LLM create the SQL query. In our tests, the large LLMs also succeeded in converting German-language prompts into correct SQL with the English table structure, but the success rate for complex queries was significantly lower.

But that was just the beginning. In the next section, we'll ask more difficult questions and have them answered by different language models.

12.7.3 Sample Queries

Our program starts quite quickly (in Python, "quickly" always leaves room for interpretation), but we don't want to restart it for every query. Using a `while` loop in the program, we constantly request new prompts:

```
from prompt_toolkit import PromptSession

session = PromptSession()
try:
    while True:
        q = session.prompt("Query our viel-falter
database: ")
        if q == "exit":
            break
        answer(q)
except (KeyboardInterrupt, EOFError):
    print("bye")
```

The `PromptSession` function from the `prompt_toolkit` (`pip install prompt-toolkit`) saves a list of queries that have already been submitted. You can scroll through older entries using the `[Ctrl]+[P]` shortcut and even search in these entries via `[Ctrl]+[R]`. The program can be terminated by entering `exit` or using `[Ctrl]+[C]` (`KeyboardInterrupt`) or `[Ctrl]+[D]` (`EOFError`). Now we want the AI tool to answer a few more difficult questions.

Prompt: How many surveys were conducted in 2024 and on how many different sites?

Based on the SQL query results, in 2024:

1. A total of 586 surveys were conducted.
2. These surveys were carried out across 184 different sites.

The answer sounds good, but can we believe it? It would be interesting to see the SQL query that led to this result. Fortunately, we don't have to do much additional programming, but only activate logging for the LlamaIndex library. We use the same command-line parameters as in the previous example ([Section 12.6](#)) to activate logging. The SQL commands are displayed if the log level is set to `DEBUG`. To do this, we start the application with the `--debug` switch.

The log messages contain detailed logs of the connections to the LLM (we've chosen Claude 3.5 Sonnet) and the prompts that LlamaIndex generates and sends to the LLM. The rather detailed prompt that is used to generate the SQL query is interesting:

LlamaIndex-Prompt: Given an input question, first create a syntactically correct SQLite query to run, then look at the results of the query and return the answer. You can order the results by a relevant column to return the most interesting examples in the database. Never query for all the columns from a specific table, only ask for a few relevant columns given the question. Pay attention to using only the column names that you can see in the schema description. Be careful not to query for columns that don't exist. Pay attention to which column is in which table. Also, qualify column names with the table name when needed. You are required to use the following format, each taking one line:

Question: Question here

SQLQuery: SQL Query to run

SQLResult: Result of the SQLQuery Answer

The prompt is then extended by the structure of all tables we've specified in the program (the `tables` variable) and, of course, by the question asked. In the further course of the communication, we see the suggested SQL query:

Predicted SQL query:

```
SELECT COUNT(*) AS total_surveys,  
       COUNT(DISTINCT site_id) AS distinct_sites  
FROM surveys WHERE date LIKE '2024%'
```

A date comparison using LIKE may seem a little unconventional for SQL purists, but in SQLite, the query works as desired. We find the answer in the log output as SQL response: [(586, 184)]. The nicely formulated answer then generates the LLM again, which formulates English sentences from the numbers and the context. Thus, Claude Sonnet is doing a pretty good job. Now we wanted to find out how the local models in an Ollama server fare in this regard. Meta's current open-source model, llama3.1, initially gave us little hope:

Based on the query response, it appears that there were no surveys conducted in 2024.

Therefore, the synthesized response would be:

"No surveys were conducted in 2024. Additionally, because there were no surveys, there are also no different sites associated with these surveys."

The analysis of the SQL query immediately revealed the problem:

```
SELECT COUNT(DISTINCT site_id) AS total_sites,  
       COUNT(*) AS total_surveys  
FROM surveys WHERE date = '2024';
```

Most of the query was actually correct, only the date field wasn't analyzed correctly, which is disappointing. We immediately tried the query again and were somewhat surprised that llama3.1 suddenly had a solution ready, but with a different result than the AI from Anthropic (Claude) before. (Llama 3.1 carried out the highlighting in the result.)

According to our records, **76 different sites** were surveyed in **2024**, with a total of **500 surveys** conducted across these locations.

Debug logging helped us solve this mystery as well. The AI tool had tried to do too much here and linked the list of species found (`speciesList`) to the `surveys` table. But there are also surveys in which no butterflies are found, and that is also a result (the fact that the differences are so great is also due to the fact that not all surveys are carried out at species level, but that would go too far here).

Predicted SQL query:

```
SELECT COUNT(DISTINCT T1.site_id) AS num_sites,
       COUNT(T1._id) AS num_surveys
FROM surveys AS T1
      INNER JOIN speciesList AS T2 ON T1._id =
T2.survey_id
WHERE STRFTIME('%Y', T1.date) = '2024'
```

The story got a little scary when we ran the query again and got the right result, which the Claude AI tool had provided straight away. The three tests took place within one minute, and the Ollama server had no other computing tasks. Perhaps the `llama3.1` model isn't good enough for this task. We're also testing the `deepseek-coder-v2` model in Ollama. However, our initial confidence was dampened again with more complex queries, as we were faced with the same problem of answers being incorrect and different for each query.

Our impression was that these smaller local models can't handle complex Text-to-SQL tasks properly. By the time you hold this book in your hands, these models will certainly be much more advanced and may be able to execute Text-to-SQL well locally.

Claude AI and OpenAI's `gpt-4o`, on the other hand, have repeatedly surprised us. In the following query, how many individuals of a specific species were found in a region in one year, all four tables of our SQLite database had to be joined in the SQL query, and the numeric `count` field in one of the tables had to be summed up. The `nuts2` database field is derived from the European classification of territorial units (<https://ec.europa.eu/eurostat/web/nuts>).

Prompt: How many individuals of species “Admiral” were found in surveys in 2024 in the Nuts2 region of Vorarlberg?

Based on the SQL query results, 4 individuals of the species “Admiral” were found in surveys conducted in 2024 in the NUTS2 region of Vorarlberg.

We couldn’t have written the SQL query used for this much better ourselves:

```
SELECT SUM(sl.count) as total_count
FROM speciesList sl
JOIN surveys s ON sl.survey_id = s._id
JOIN species sp ON sl.species_id = sp._id
JOIN sites st ON s.site_id = st._id
WHERE sp.german = 'Admiral'
      AND strftime('%Y', s.date) = '2024'
      AND st.nuts2 = 'Vorarlberg'
```

By playing around with the database, we also found incorrect entries by chance. The search for all existing Nuts2 regions led to a surprising result.

Prompt: How many different Nuts2 regions are in the database and list them?

Based on the SQL query results, there are 11 different Nuts2 regions in the database. The regions are:

1. Tirol
2. Vorarlberg
3. Südtirol
4. Niederösterreich
5. Salzburg
6. Oberösterreich
7. Kärnten
8. Steiermark
9. Burgenland
10. Wien

It's worth noting that there appears to be an empty entry in the list (indicated by the double comma between Burgenland and Wien), which could represent a null or empty value in the database. This empty entry is counted as one of the 11 distinct regions, even though it doesn't have a name.

However, the AI tool immediately explains where the 11th entry, which doesn't appear in the list, comes from: there is one entry where the region is obviously empty. Without a little knowledge of the database structure, it will be difficult to find the incorrect entry. The SQL query that led to the result also helps us here:

```
SELECT COUNT(DISTINCT nuts2) AS count,  
        GROUP_CONCAT(DISTINCT nuts2) AS regions  
FROM sites
```

This is an entry in the sites table. Now we can ask the AI tool which entry in this table it is:

Prompt: Show me sites not assigned to a Nuts2 region. Include database IDs for these sites.

Based on the query results, there is one site that isn't assigned to a NUTS2 region. The details are as follows:

Database ID: 65f6a6f616634be52c735711

Name: [No name provided]

It's worth noting that while the database ID is present, the name field appears to be empty for this site. This could mean either the site doesn't have a name recorded in the database, or there might be an issue with the data entry for this particular site.

The incorrect entry was quickly corrected using the database ID.

12.7.4 Conclusion

The Text-to-SQL concept has great potential for us. This concept could save database administrators work, especially in environments where people without a technical background want to sporadically request different key figures from a central database. Will this make the database admins superfluous? Certainly not at the moment.

The problem of not being able to rely on the AI tool's answers, even with Text-to-SQL, makes the system unsuitable for a business-critical area. But the same also applies to hallucinating models for other tasks.

Another limiting factor is currently the number of tokens an LLM can process in context. If the SQL queries contain too much data, the LLM can no longer process the responses. However, we also expect rapid development in this field. It will definitely be exciting to see where this technology can establish itself in the future.